影像處理 Image Processing

大學智慧機器人學程 | 高中 AI 機器人課程 | 台灣自編大學教科書

編者：李世淵(機器人叫獸)|助教：李天宇、李宇晴

第 2 章 開發環境與程式基礎

本章學習目標

- 建立 Python 與 OpenCV 的開發環境，理解虛擬環境與套件管理的必要性。
- 掌握 NumPy 陣列運算，理解「影像就是陣列」這個貫穿全書的核心觀念。
- 熟練 OpenCV 的影像讀取、顯示、儲存與基本操作，並避開 BGR 等常見陷阱。
- 運用 ROI、切片與遮罩對影像的局部區域進行操作。
- 比較端上、近端邊緣、雲端三種 AI 辨識架構，並建立 ESP32-CAM 與 Jetson 雙平台環境。
- 養成除錯、查閱文件與版本管理的工程習慣。

叫獸開講|OpenCV 的誕生：一個「不要再重造輪子」的決定

1998 年，英特爾研究院的一位研究員 Gary Bradski 有了一個念頭。當時他觀察到一個奇怪的現象：電腦視覺這個領域，每一個實驗室、每一間公司，都在重複寫著同樣的東西——讀取影像、做高斯模糊、找邊緣、算直方圖。這些基礎功能人人都需要，卻沒有人共用;研究生入學的第一年，往往就耗在重寫前人早已寫過無數次的程式碼上。整個領域的能量，大量浪費在重造輪子。

Bradski 的構想是：打造一套共用的、經過最佳化的電腦視覺函式庫，讓研究者能站在同一個基礎上，把時間花在真正的創新上。他在英特爾內部組建了團隊，由 Vadim Pisarevsky 擔任技術主管，並得到英特爾俄羅斯團隊與效能函式庫團隊的大力支援。

但真正關鍵的轉折點在 1999 年。英特爾原本的計畫是做一套閉源的商業產品，名字叫 CVL(Computer Vision Library)。是 Bradski 力主開源——他認為只有開放，才能真正達成「不要再重造輪子」的目標，也才能讓整個領域一起前進。對一家以販售技術為業的晶片巨頭來說，把辛苦開發的程式碼免費送出去，是個相當大膽的決定，而英特爾接受了。專案也因此改名為 OpenCV，靈感來自當時已相當流行的 OpenGL。

2000 年 6 月，OpenCV 的第一個公開版本在電腦視覺頂級會議 CVPR 上發表，獲得與會者高度讚譽。它的目標在早期文件中寫得很清楚：提供不只是開放、而且是經過最佳化的基礎視覺程式碼，讓大家不必再重造輪子;透過共用的基礎設施傳播視覺知識;並以不強制開源的授權條款，讓商業應用也能自由採用。這最後一點尤其重要——正是這個寬鬆的授權，讓 OpenCV 得以進入無數的商業產品(對照第 13 章 SIFT 專利的故事，你會更能體會授權條款的份量)。

接下來的故事你已經身在其中。2005 年，史丹佛大學的無人車 Stanley 使用 OpenCV，贏得了美國國防高等研究計畫署的無人車大挑戰，在莫哈維沙漠中自主行駛超過兩百公里;2006 年 1.0 正式版問世;此後歷經 Willow Garage、Itseez 的接手維護，加入了 C++ 與 Python 介面、GPU 加速、深度學習模組。今天，OpenCV 收錄超過兩千五百個演算法，累積數千萬次下載，是全世界電腦視覺工作者共同的工具箱——包括你即將在本章安裝的這一套。

叫獸想從這個故事裡點出兩件事。第一，你在這門課裡寫的每一行 OpenCV 程式，背後都站著二十多年來成千上萬人的貢獻。這正是第 18 章 4P 中「同儕」(Peers)在全球尺度上的展現——開源社群本身，就是人類最大規模的協作學習實驗。第二，Bradski 當年那個「不要再重造輪子」的判斷，今天依然適用於你：寫程式時，先問問「這個功能是不是已經有人寫得比我好了?」把時間留給真正需要你創造的那一部分。

思考題：英特爾把辛苦開發的程式碼免費開源，短期看是「損失」，長期卻

讓公司在電腦視覺生態系中佔據了核心位置。你能想到其他「開放反而獲益」的例子嗎?這對你思考「知識該不該分享」有什麼啟發?

2.1 開發環境的建立

2.1.1 為什麼選 Python 與 OpenCV

本書選用 Python 搭配 OpenCV 作為主要工具，理由有四：其一，Python 語法簡潔，讓你能專注在影像處理的觀念而非語言細節;其二，OpenCV 是業界標準，學會它等於學會了一項可直接用於職場的技能;其三，Python 擁有最完整的科學運算與深度學習生態系(NumPy、Matplotlib、PyTorch、Ultralytics)，第 14 至 16 章會大量使用;其四，同樣的程式碼稍加調整就能在 Jetson 等邊緣裝置上執行(第 17 章)。

需要提醒的是：OpenCV 的核心是以 C++ 撰寫並高度最佳化，Python 只是它的介面。這代表你享受了 Python 的易用性，卻仍能獲得接近 C++ 的執行速度——前提是你使用 OpenCV 或 NumPy 的向量化運算，而非用 Python 的迴圈逐像素處理(2.3.3 節會用實測數據說明這個差異有多大)。

2.1.2 虛擬環境：為什麼不要直接安裝

初學者常直接用系統的 Python 安裝套件，這會在專案變多時造成災難：A 專案需要某套件的舊版本、B 專案需要新版本，兩者無法共存。虛擬環境(virtual environment)為每個專案建立獨立的套件空間，彼此互不干擾。這是專業開發的基本習慣，請從第一天就養成。

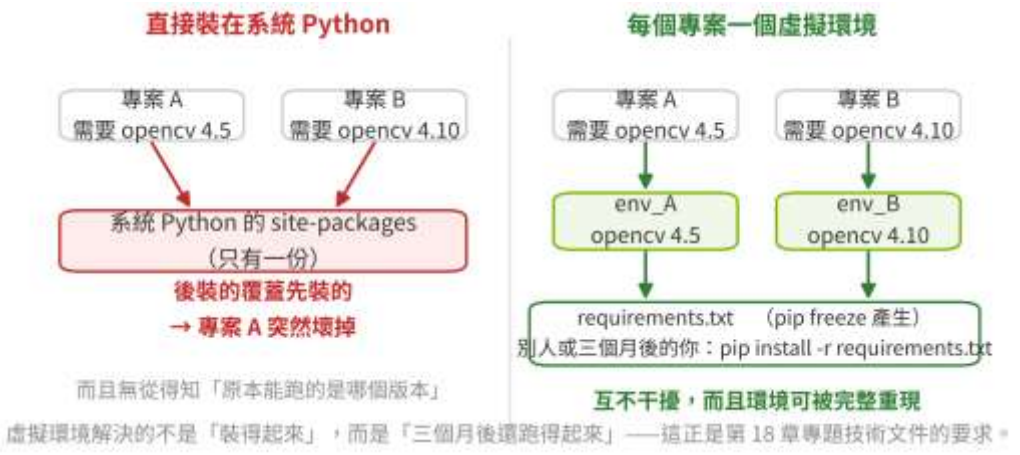


圖 2-1　虛擬環境解決的是什麼問題

程式 2-1 建立虛擬環境與安裝套件

```
# --- 建立虛擬環境(以 Python 內建的 venv 為例) ---
python -m venv cv_env

# 啟動(Windows)
cv_env\Scripts\activate
# 啟動(macOS / Linux)
source cv_env/bin/activate

# --- 安裝本課程所需套件 ---
pip install opencv-python        # OpenCV 主套件
pip install opencv-contrib-python  # 含額外模組(第 12 章追蹤器需要)
pip install numpy matplotlib     # 陣列運算與繪圖

# --- 記錄環境,方便他人重現(團隊協作與第 18 章專題必備) ---
pip freeze > requirements.txt
# 他人只需:pip install -r requirements.txt

# --- 離開虛擬環境 ---
deactivate
```

請注意 requirements.txt 這個檔案。它記錄了專案所需的所有套件與版本，讓別人(或三個月後的你)能完整重現你的環境。第 18 章期末專題的技術文件要求「另一個人照著做能重現你的成果」，這個檔案就是關鍵的一環。

2.1.3 驗證安裝與開發工具

程式 2-2 驗證環境安裝

```
import cv2
import numpy as np
import sys

print("Python 版本:", sys.version)
```

```
print("OpenCV 版本:", cv2.__version__)
print("NumPy 版本 :", np.__version__)

# 測試:建立一張純色影像並顯示
img = np.zeros((300, 400, 3), dtype=np.uint8)   # 全黑影像
img[:] = (0, 185, 118)                          # BGR:綠色
cv2.putText(img, "Hello OpenCV!", (60, 160),
            cv2.FONT_HERSHEY_SIMPLEX, 1, (255, 255, 255), 2)
cv2.imshow("test", img)
cv2.waitKey(0)              # 按任意鍵關閉
cv2.destroyAllWindows()
```

開發工具方面，建議三選一：Jupyter Notebook 適合探索與教學(能逐格執行、即時看結果，本課程的 Lab 很適合);VS Code 適合開發完整專案(有除錯器、Git 整合);Google Colab 則適合沒有本機環境或需要免費 GPU 的場合(第 15、16 章訓練模型時特別有用)。三者可以並用，依任務選擇。



圖 2-2　Jupyter、VS Code、Colab 的適用場合

2.2 影像就是陣列：NumPy 基礎

這一節是全書最重要的觀念之一。如果你只記得本章的一句話，請記得這一句：對電腦而言，一張影像就是一個數字陣列。

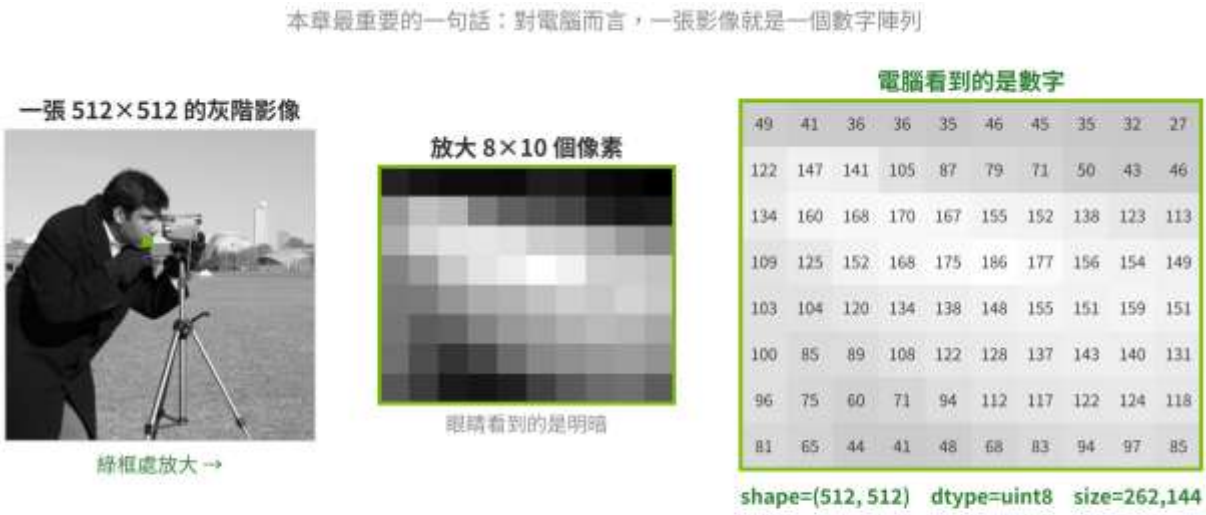


圖 2-3　把影像放大到看見數字：眼睛看明暗，電腦看數值

2.2.1 影像的資料結構

OpenCV 讀進來的影像，型別是 NumPy 的 ndarray。灰階影像是二維陣列，形狀為(高度， 寬度);彩色影像是三維陣列，形狀為(高度， 寬度， 通道數)。每個元素的資料型別通常是 uint8(無號八位元整數)，數值範圍 0 到 255——這正是第 3 章要深入探討的量化與位元深度。

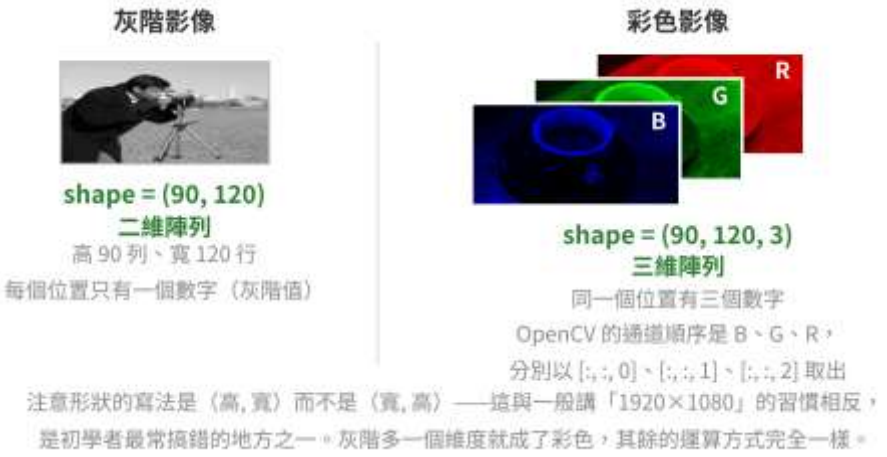


圖 2-4　灰階是二維陣列，彩色是三維陣列

請特別留意兩個容易搞混的順序。第一，陣列的索引順序是「先列後行」，也就是 img[y, x]，與數學上習慣的 (x, y) 相反;第二，OpenCV 的彩色通道順序是 BGR 而非

RGB(2.3.2 節會說明這個歷史包袱)。這兩點是初學者最常見的錯誤來源。



圖 2-5　初學者最常踩的兩個坑：索引順序與通道順序

程式 2-3 影像的陣列結構

```
import cv2, numpy as np

gray = cv2.imread("photo.jpg", cv2.IMREAD_GRAYSCALE)
color = cv2.imread("photo.jpg")            # 預設讀成彩色(BGR)

print("灰階形狀:", gray.shape)             # (高, 寬)
print("彩色形狀:", color.shape)            # (高, 寬, 3)
print("資料型別:", color.dtype)            # uint8
print("總像素數:", color.size)             # 高 × 寬 × 3

# 存取單一像素(注意:先 y 後 x!)
y, x = 100, 250
print("該點灰階值:", gray[y, x])
b, g, r = color[y, x]                      # BGR 順序,不是 RGB
print(f"該點色彩:B={b}, G={g}, R={r}")

# 修改像素
color[y, x] = (0, 0, 255)                  # 設為紅色(BGR)
```

2.2.2 建立與初始化影像

程式 2-4 以 NumPy 建立測試影像

```
import numpy as np

black = np.zeros((300, 400, 3), dtype=np.uint8)        # 全黑
white = np.ones((300, 400, 3), dtype=np.uint8) * 255   # 全白
```

```
gray  = np.full((300, 400), 128, dtype=np.uint8)        # 中灰(單通道)
noise = np.random.randint(0, 256, (300, 400),
                          dtype=np.uint8)               # 隨機雜訊

# 漸層(用於測試第 5 章的灰階轉換)
ramp = np.tile(np.arange(256, dtype=np.uint8), (100, 1))

# 棋盤格(用於測試第 4 章相機校正與第 6 章頻譜)
board = np.indices((320, 320)).sum(axis=0) // 40 % 2
board = (board * 255).astype(np.uint8)
```

自己合成測試影像是一個非常實用的技巧：當演算法出錯時，用一張你完全知道內容的簡單影像(純色塊、漸層、棋盤格)去測試，遠比用複雜的自然照片容易找出問題。第 6 章 Lab 的「頻譜偵探」就大量使用這個技巧。

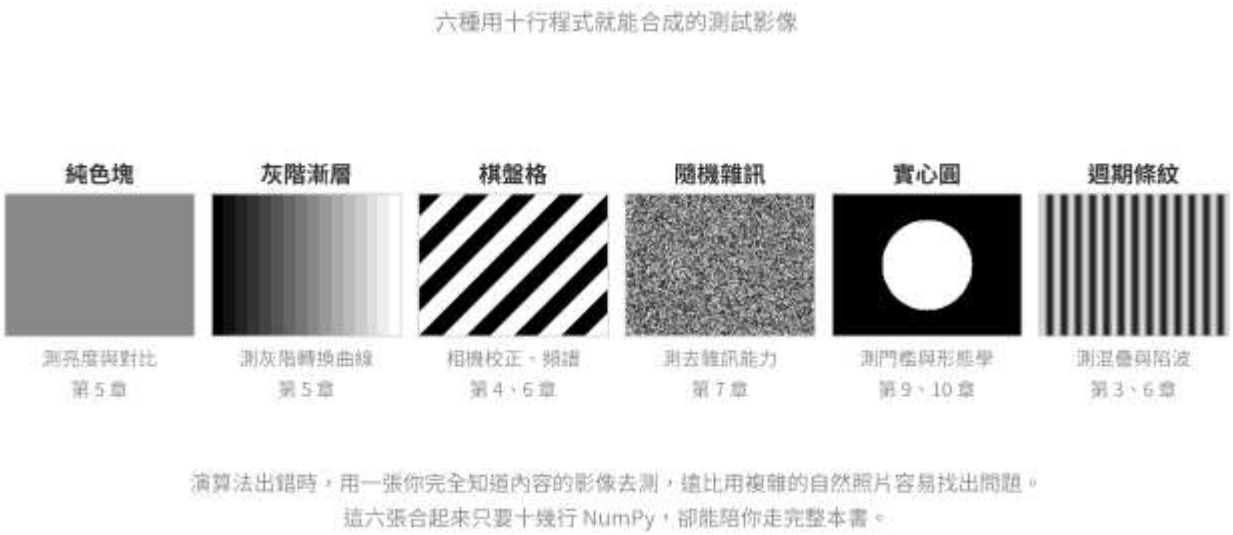


圖 2-6　六種可以用十行程式合成的測試影像

2.2.3 向量化運算：為什麼不要用迴圈

NumPy 的威力在於向量化——對整個陣列一次運算，而非逐元素迴圈。這不只是寫法簡潔的問題，而是效能的天壤之別：NumPy 的運算底層以 C 實作並最佳化，而 Python 的迴圈每次疊代都有直譯器的額外開銷。



圖 2-7　向量化與 Python 迴圈的實測效能差距

程式 2-5 向量化 vs. 迴圈的效能比較

```
import cv2, numpy as np, time

img = cv2.imread("photo.jpg", cv2.IMREAD_GRAYSCALE)
h, w = img.shape

# --- 壞方法:Python 迴圈逐像素(慢) ---
t0 = time.time()
out1 = img.copy()
for y in range(h):
    for x in range(w):
        out1[y, x] = 255 - img[y, x]        # 負片
print(f"迴圈:{time.time()-t0:.3f} 秒")

# --- 好方法:向量化(快數百倍) ---
t0 = time.time()
out2 = 255 - img                            # 一行搞定
print(f"向量化:{time.time()-t0:.5f} 秒")

print("結果相同:", np.array_equal(out1, out2))

# 常見向量化操作
brighter = np.clip(img.astype(np.int16) + 50, 0, 255).astype(np.uint8)
mask = img > 128                            # 布林陣列(第 9 章門檻分割的基礎)
print("亮部像素比例:", mask.mean())
```

請務必實際跑一次程式 2-5，親眼看看那個倍數差距。這個習慣在第 17 章邊緣部署時格外重要——在算力有限的 Jetson 或樹莓派上，一個寫成迴圈的前處理步驟，就足以讓整個即時系統掉影格。

另一個常見陷阱是溢位：uint8 的範圍是 0 到 255，若直接對它做加法，超過 255

會「繞回」變成小數字(例如 250 + 50 得到 44)，而非停在 255。正確做法是先轉成較大的型別、運算後再用 np.clip 限制範圍——程式 2-5 的 brighter 就是示範。第 5 章的亮度調整會反覆用到這個技巧。



圖 2-8　uint8 溢位：250 加 50 為什麼會變成 44

2.2.4 深入：廣播機制——NumPy 最常被低估的一招

到目前為止我們寫的 img * 1.2、255 - img，都用到了一個沒有明講的機制：廣播（Broadcasting）。它是 NumPy 在形狀不同的陣列之間自動對齊維度的規則，也是「影像就是陣列」這個觀念真正發揮威力的地方。

規則只有一條，而且只要三十秒就能記住：把兩個陣列的形狀從最後一個維度往前對齊比較，每一組維度必須相等，或其中之一為 1，否則無法廣播。為 1 的那一維會被「複製」到另一邊的長度——但這個複製是虛擬的，NumPy 並不會真的配置一塊大記憶體，而是用步幅為零的方式反覆讀取同一份資料，所以既省空間又快。

這條規則在影像處理上有三種典型用法，請務必對照形狀理解。其一是純量：img * 1.2，純量視為形狀 ()，可廣播到任何形狀，效果是整體調整亮度。其二是逐通道：對一張 (480, 640, 3) 的彩色影像乘上一個形狀為 (3,) 的陣列，例如 img *

np.array([0.9, 1.0, 1.2])，三個係數分別作用在 B、G、R 三個通道上——這正是白平衡與色溫調整的做法（第 8 章）。其三是逐列或逐行：一個形狀 (480, 1) 的陣列可以廣播到 (480, 640)，效果是「每一列乘上不同的係數」，這是製作漸層遮罩與暈影（vignette）效果的標準手法。

最常見的錯誤是維度對不齊。例如想對 (480, 640, 3) 的影像逐列加權，若直接乘上形狀 (480,) 的陣列會失敗，因為對齊是從最後一維開始：480 與 3 既不相等、也都不是 1。正解是先把它變成 (480, 1, 1)，用 w[:, None, None] 或 w.reshape(-1, 1, 1) 即可。一個實用的除錯習慣是：遇到廣播錯誤時，先把兩邊的 shape 印出來，右對齊寫在紙上比對，問題通常一眼就看得出來。

最後提醒一個與第 14 至 16 章直接相關的應用：深度學習的輸入正規化 (img - mean) / std，其中 mean 與 std 都是形狀 (3,) 的陣列，靠的就是逐通道廣播。許多初學者在這一步把形狀搞錯卻不自知——因為程式不會報錯，只是模型永遠訓練不起來。

2.3 OpenCV 基本操作

2.3.1 讀取、顯示與儲存

程式 2-6 影像的讀取、顯示與儲存

```
import cv2

# --- 讀取 ---
img  = cv2.imread("photo.jpg")                    # 彩色(BGR)
gray = cv2.imread("photo.jpg", cv2.IMREAD_GRAYSCALE)
raw  = cv2.imread("photo.png", cv2.IMREAD_UNCHANGED)  # 含 alpha 通道

# 務必檢查是否讀取成功(路徑錯誤時 imread 回傳 None 而不報錯!)
if img is None:
    raise FileNotFoundError("找不到影像,請檢查路徑")

# --- 顯示 ---
cv2.imshow("window title", img)
key = cv2.waitKey(0)          # 0 = 無限等待;數字 = 等待毫秒數
cv2.destroyAllWindows()

# --- 儲存(副檔名決定格式與壓縮方式,第 11 章詳述) ---
cv2.imwrite("output.png", img)                    # 無損
```

```
cv2.imwrite("output.jpg", img,
            [cv2.IMWRITE_JPEG_QUALITY, 95])       # 有損,可設品質
```

三個實務提醒：第一，imread 找不到檔案時不會拋出例外，而是安靜地回傳 None，若不檢查會在後續操作時出現難以理解的錯誤訊息——請養成檢查的習慣。第二，中文路徑在某些系統上會導致讀取失敗，建議檔名與資料夾都用英文。第三，waitKey 的回傳值是按鍵碼，可用於互動控制(第 8、12 章的 Lab 會用到)。

2.3.2 BGR 之謎：一個歷史包袱

OpenCV 使用 BGR 而非 RGB 的通道順序，是初學者最常踩的坑——用 Matplotlib 顯示 OpenCV 讀進來的影像時，紅色和藍色會對調。這個設計來自 OpenCV 早期(2000 年代初)的歷史背景：當時許多相機硬體與 Windows 的點陣圖格式採用 BGR 排列，為了效能而直接沿用。等到 RGB 成為主流，OpenCV 已有大量既有程式碼依賴 BGR，改動的代價太高，只好一路保留至今。



圖 2-9　BGR 與 RGB：同一個陣列，兩種讀法

這個小小的歷史包袱其實是一堂很好的工程課：早期的設計決定會長期影響一個系統，即使當初的理由早已消失。你在第 18 章的專題中做架構決定時，不妨想想這件事。

程式 2-7 BGR 與 RGB 的轉換

```
import cv2, matplotlib.pyplot as plt

img = cv2.imread("photo.jpg")        # BGR

# 用 Matplotlib 顯示時務必轉換,否則紅藍顛倒
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.axis("off"); plt.show()

# 通道的分離與合併
b, g, r = cv2.split(img)
merged  = cv2.merge([b, g, r])

# 用 NumPy 切片更快(向量化思維)
blue_channel = img[:, :, 0]
rgb = img[:, :, ::-1]                # 反轉通道順序即 BGR→RGB
```

2.3.3 ROI、切片與遮罩

ROI(Region of Interest，感興趣區域)是只對影像的某個區域進行操作的技巧，在 NumPy 中用切片即可完成。它在整本書中反覆出現：第 9 章車道線偵測只處理路面梯形區、第 10 章人臉偵測後只在臉部區域找眼睛、第 13 章裁切車牌區域做字元分割。



圖 2-10　ROI 就是切片：座標範圍與語法的對照

程式 2-8 ROI、切片與遮罩

```
import cv2, numpy as np

img = cv2.imread("photo.jpg")
```

```
# --- ROI:用切片取出區域(注意:先 y 後 x) ---
roi = img[100:300, 200:500]          # y 從 100 到 300,x 從 200 到 500
cv2.imwrite("roi.jpg", roi)

# 對 ROI 做處理後貼回(注意:切片是「檢視」不是複製!)
img[100:300, 200:500] = cv2.GaussianBlur(roi, (21, 21), 0)

# 若不想影響原圖,務必用 copy()
roi_safe = img[100:300, 200:500].copy()

# --- 遮罩:對不規則區域操作 ---
mask = np.zeros(img.shape[:2], dtype=np.uint8)
cv2.circle(mask, (320, 240), 120, 255, -1)      # 白色實心圓
masked = cv2.bitwise_and(img, img, mask=mask)   # 只保留圓內

# 遮罩也可用布林索引(NumPy 風格)
img[mask == 255] = (0, 255, 0)                  # 圓內塗綠
```

程式中「切片是檢視不是複製」這一點請特別留意：NumPy 的切片回傳的是原陣列的一個視圖，修改它會直接改到原影像。這既是效能優勢(不必複製資料)，也是常見的 bug 來源。需要獨立副本時，務必呼叫 copy()。



圖 2-11　切片是「檢視」不是「複製」的實測對照

2.3.4 深入：記憶體佈局——為什麼切片不用複製

前一段說「切片是檢視不是複製」，但沒有解釋它為什麼做得到。答案藏在 NumPy 描述陣列的方式裡，而這個機制會在本書反覆出現，值得花兩頁弄懂。

一個 NumPy 陣列由三樣東西描述：一塊連續的記憶體、一個形狀（shape），

以及一組步幅（strides）——步幅記錄「在這一維上前進一格，記憶體位址要跳幾個位元組」。以一張 (480, 640, 3) 的 uint8 彩色影像為例，步幅是 (1920, 3, 1)：往下一列要跳 640 × 3 = 1920 個位元組，往右一行跳 3 個，換一個通道跳 1 個。這也印證了彩色影像在記憶體中是以 BGRBGRBGR… 交錯排列的，而不是三個通道各自成塊。

有了步幅，切片就只是「換一組起點、形狀與步幅」而已，完全不必搬動資料——這就是 img[100:300, 200:420] 幾乎零成本的原因。同理，img[:, :, ::-1] 做 BGR 轉 RGB 只是把最後一維的步幅改成 −1，比 cv2.cvtColor 更快;img.T 轉置也只是把前兩維的步幅互換。

代價是這些結果不再是「C 連續」的——也就是說，元素在記憶體中的排列順序不再與索引順序一致。純粹在 NumPy 之內運算時這沒有問題，但 OpenCV 的底層是 C++ 實作，多數函式要求輸入連續排列，拿到非連續的陣列時可能拋出錯誤，或由綁定層偷偷複製一份（於是你省下的時間又還回去了）。用 img.flags["C_CONTIGUOUS"] 可以檢查，用 np.ascontiguousarray() 可以明確複製一份連續的。

實務上的判準很簡單：只在 NumPy 之內運算時，盡量用切片與步幅操作，省時省記憶體;要把結果交給 OpenCV、寫入檔案、或送進深度學習框架時，先確保它是連續的。這個觀念在第 12 章處理視訊（每秒三十張影格都要搬一次就來不及了）與第 17 章的邊緣部署（記憶體頻寬是瓶頸）時格外重要。

2.4 三種 AI 辨識架構與雙平台

本節建立本書實作的硬體框架。這三種架構會在第 10、16、17 章反覆出現，是理解「演算法該跑在哪裡」的關鍵。

2.4.1 三種架構

1. 端上運算(On-device)：在感測裝置本身完成運算，例如 ESP32-CAM 直接執行輕量的人臉偵測。優點是延遲最低、不需網路、資料不出裝置(隱私最佳)、功耗低;限制是算力極為有限。
2. 近端邊緣運算(Edge)：在鄰近的較強裝置上運算，例如產線旁的 Jetson 或工業電腦處理多台相機的影像。它在算力與即時性之間取得平衡，資料不出廠區，是機器人與工業視覺的主力架構。
3. 雲端運算(Cloud)：把資料上傳到遠端資料中心處理。算力最強、適合訓練大模型與複雜分析，但延遲高、需要網路、且涉及資料外流的隱私考量。

關鍵觀念是三者可以分層協作。第 10 章的門禁案例就是典型：ESP32-CAM 在端上判斷「有沒有人臉」(高頻、低算力、保護隱私)，偵測到才把影像送到雲端做「是誰」的辨識(低頻、高算力)。第 17 章會用完整的比較表深入這個主題。



圖 2-13　端上、近端邊緣、雲端三種架構的取捨

2.4.2 雙平台介紹

本書所有實作圍繞兩個平台展開，對應上述架構：

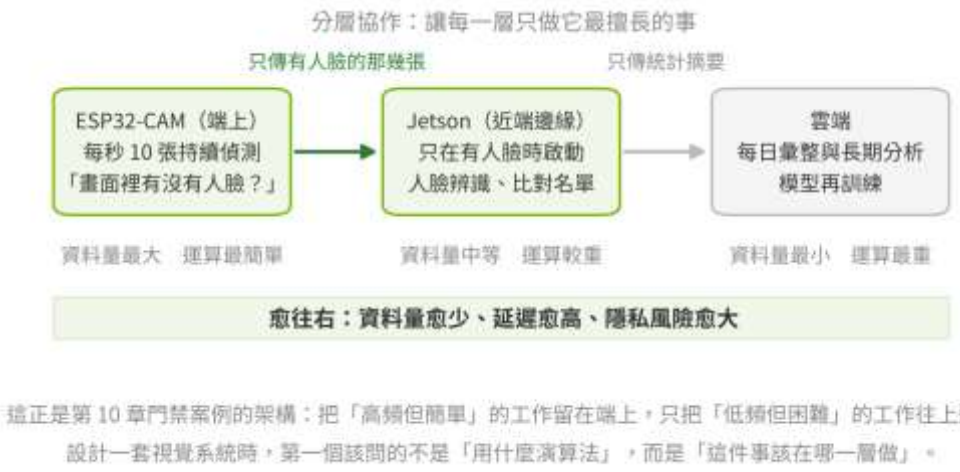


圖 2-14　分層協作：門禁案例的資料流

比較面向	ESP32-CAM(入門)	NVIDIA Jetson(進階)
定位	會看的感測節點	機器人的視覺大腦
對應架構	端上運算	近端邊緣運算
開發方式	Arduino IDE 或 ESP-IDF(C/C++)	Linux + Python + CUDA(同桌機)
可跑的任務	移動偵測、顏色追蹤、Haar 人臉偵測、影像串流	YOLO 即時偵測、分割、姿態、SLAM、ROS 2
本書出現章節	第 8、10、12、17 章 Lab	第 15-17 章與期末專題

表 2-1 雙平台比較

學習策略上，建議的流程是：先在個人電腦上用 Python 與 OpenCV 把演算法邏輯驗證清楚，再移植到目標平台。這個「先驗證邏輯、再移植硬體」的工作方式，是嵌入式與機器人開發的標準做法——因為在電腦上除錯遠比在裝置上容易。

2.4.3 ESP32-CAM 快速上手

ESP32-CAM 最常見的用法是作為網路攝影機：燒錄官方的 CameraWebServer 範例後，它會提供一個串流網址，而 OpenCV 可以直接讀取這個網址，把它當成一台普通的相機。這讓你能在電腦上用 Python 開發，卻使用 ESP32-CAM 作為影像來源——本書多章的 Lab 都採用這個模式。

程式 2-9 以 OpenCV 讀取 ESP32-CAM 串流

```
import cv2

# ESP32-CAM 燒錄 CameraWebServer 範例後取得的串流網址
STREAM_URL = "http://192.168.1.50:81/stream"

cap = cv2.VideoCapture(STREAM_URL)      # 與讀取本機相機的用法完全相同
if not cap.isOpened():
    raise ConnectionError("無法連線,請確認 ESP32-CAM 與電腦在同一網路")

while True:
    ok, frame = cap.read()
    if not ok:
        break

    # === 此處可套用本書任何一章的處理 ===
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    cv2.imshow("ESP32-CAM", frame)
    if cv2.waitKey(1) == 27:            # ESC 離開
        break

cap.release()
cv2.destroyAllWindows()
```

2.5 工程習慣的養成

這一節談的不是影像處理，而是讓你在往後十六章(以及日後職涯)少走彎路的習慣。它們看似瑣碎，卻是區分「會寫程式」與「會做專案」的關鍵。

2.5.1 除錯的基本功

影像處理的除錯有其特殊性：錯誤往往不會拋出例外，而是「結果看起來怪怪的」。三個實用技巧：其一，隨時印出陣列的 shape 與 dtype——大多數錯誤都源於形狀不符或型別溢位;其二，把中間結果存檔或顯示出來，不要一路算到底才看結果(所謂「視覺化除錯」);其三，用自己合成的簡單影像(2.2.2 節)測試，你知道正確答案應該長什麼樣。



圖 2-15　影像處理的除錯：每個階段都印出 shape 與值域

程式 2-10 除錯輔助函式

```
import cv2, numpy as np

def debug_info(name, arr):
    """印出陣列的關鍵資訊,除錯必備"""
    print(f"{name:12s} shape={arr.shape} dtype={arr.dtype} "
          f"min={arr.min()} max={arr.max()} mean={arr.mean():.1f}")

img = cv2.imread("photo.jpg", cv2.IMREAD_GRAYSCALE)
debug_info("原圖", img)

blur = cv2.GaussianBlur(img, (5, 5), 0)
debug_info("模糊後", blur)

edges = cv2.Canny(blur, 80, 160)
debug_info("邊緣", edges)

# 視覺化除錯:把各階段並排顯示
stacked = np.hstack([img, blur, edges])
cv2.imwrite("debug_stages.jpg", stacked)
```

2.5.2 查閱文件與求助

沒有人記得住 OpenCV 兩千多個函式的所有參數。專業工程師的能力不在於「記得多少」，而在於「多快能找到並理解」。建議的順序：先看官方文件(docs.opencv.org，有完整的參數說明與範例)、再看官方教學(tutorial 章節)、最後才是搜尋引擎與問答網站。使用 AI 助手時也請記得驗證——它可能給你舊版本的 API

或看似合理但錯誤的參數。

一個實用技巧：在 Python 中直接用 help(cv2.GaussianBlur) 或在 Jupyter 中用問號查詢函式簽名，比切換到瀏覽器更快。

2.5.3 版本管理與程式碼組織

從第一個 Lab 就開始用 Git。它的價值不只是備份，更在於「讓你敢於嘗試」——知道隨時能回復到能用的版本，你才會放手實驗(這正是第 18 章 Play 精神的技術基礎)。程式碼組織上，建議把常用的處理函式抽成獨立模組(例如把第 5 章的前處理管線寫成 preprocess.py)，往後各章直接引用——這既是良好習慣，也呼應叫獸開講「不要重造輪子」的精神。



圖 2-16　有版本控制，才敢放手做實驗

2.5.4 深入：從「會跑」到「可重現」

一支程式在你的電腦上跑得出結果，與這個結果能被別人（或三個月後的你）重現，是兩件完全不同的事。前者是作業的及格線，後者是第 18 章專題與日後職涯的及格線。兩者之間的差距，通常來自四個容易被忽略的細節。

第一是環境。虛擬環境加 requirements.txt 解決了套件版本（圖 2-1），但還有兩件事值得記錄：Python 本身的版本，以及作業系統——某些 OpenCV 函式在不同平台上的預設行為確實有細微差異。最省事的做法是在 README 的開頭寫下程式 2-2

的三行輸出。

第二是隨機性。只要你的流程裡出現 np.random、資料集的隨機切分、或第 15 章的權重初始化，每次執行的結果就會不同。解法是固定亂數種子（np.random.default_rng(42)），並且把種子值寫進紀錄而不是埋在程式中段。注意：固定種子是為了「可重現」，不是為了「挑一個好看的結果」——若你換了十個種子才得到想要的數字，該報告的是十次的分布，而不是最好的那一次。這是研究誠信的底線。

第三是路徑與資料。把 "C:/Users/你的名字/Desktop/photo.jpg" 寫死在程式裡，別人拿到就跑不動。做法是把所有路徑集中成檔案開頭的常數、使用相對路徑、並固定 raw / processed / output 的資料夾結構，其中 raw 永遠唯讀——覆寫原始資料是這門課唯一無法復原的錯誤。

第四是紀錄。每次執行時把參數、輸入檔名、輸出檔名與關鍵數值寫進一個簡單的文字檔或 CSV。這件事做起來只要五行程式，卻能在三週後你盯著一張效果很好的結果圖、卻完全想不起來當初參數調到多少時，救你一命。

把這四件事加上 Git，就構成了一份最小可行的「可重現」清單。它們看起來都與影像處理無關，但第 18 章評分時真正拉開差距的，往往正是這些地方。

2.6 應用案例

2.6.1 案例一：批次影像處理工具

情境：你有一資料夾的照片需要統一處理——調整尺寸、轉灰階、加浮水印、重新命名。這是影像處理最基本也最常見的實務需求。做法：用 Python 的 glob 或 pathlib 列出所有檔案，逐一讀取、處理、儲存到輸出資料夾。

這個看似簡單的案例，實際上是所有後續工作的基礎：第 14 至 16 章訓練模型前，你都需要先把原始影像批次處理成統一的尺寸與格式。及早寫好一個可重複使用的批

次處理腳本，會替你省下大量時間。



圖 2-17　批次處理：把尺寸各異的照片統一成同一規格

程式 2-11 批次影像處理

```
import cv2, glob, os

IN_DIR, OUT_DIR = "raw/", "processed/"
os.makedirs(OUT_DIR, exist_ok=True)

for i, path in enumerate(sorted(glob.glob(IN_DIR + "*.jpg"))):
    img = cv2.imread(path)
    if img is None:                          # 務必檢查
        print(f"跳過無法讀取的檔案:{path}")
        continue

    img = cv2.resize(img, (640, 480))     # 統一尺寸(第 4 章)
    cv2.putText(img, "YunTech", (10, 30),
                cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 185, 118), 2)

    out_path = os.path.join(OUT_DIR, f"img_{i:04d}.jpg")
    cv2.imwrite(out_path, img)

print("批次處理完成")
```

2.6.2 案例二：即時攝影機處理管線

情境：打開筆電相機，即時顯示原始畫面與處理後的結果並排對照。這個「讀取—影格、處理、顯示」的迴圈，是第 12 章視訊處理、第 16 章 YOLO 即時偵測、第 17 章邊緣部署的共同骨架——本書後半的所有即時應用，結構都與它相同。

建議在這個案例中加入按鍵切換功能(例如按 1 顯示原圖、按 2 顯示灰階、按 3 顯示邊緣)，它會成為你整學期最常用的實驗工具：每學一個新演算法，就把它加進這個管線，即時看效果。

程式 2-12 即時處理管線（按鍵切換模式）

```
import cv2

cap = cv2.VideoCapture(0)          # 0 = 預設相機;換成網址即可讀 ESP32-CAM
if not cap.isOpened():
    raise ConnectionError("無法開啟相機")

mode = 1                           # 目前的處理模式
NAMES = {1: "original", 2: "gray", 3: "edges", 4: "blur"}

while True:
    ok, frame = cap.read()
    if not ok:
        break

    # === 依模式處理:每學一個新演算法就在這裡加一個分支 ===
    if mode == 1:
        out = frame
    elif mode == 2:
        out = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    elif mode == 3:
        g = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
        out = cv2.Canny(cv2.GaussianBlur(g, (5, 5), 0), 70, 150)
    else:
        out = cv2.GaussianBlur(frame, (21, 21), 0)

    # 灰階結果轉回三通道,才能疊彩色文字並與原圖並排
    if out.ndim == 2:
        out = cv2.cvtColor(out, cv2.COLOR_GRAY2BGR)

    cv2.putText(out, f"[{mode}] {NAMES[mode]}", (10, 30),
                cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 185, 118), 2)
    cv2.imshow("pipeline", cv2.hconcat([frame, out]))   # 原圖與結果並排

    key = cv2.waitKey(1) & 0xFF
    if key == 27:                  # ESC 離開
        break
    elif key in (ord("1"), ord("2"), ord("3"), ord("4")):
        mode = key - ord("0")
    elif key == ord("s"):          # 按 s 存下目前這一格
        cv2.imwrite(f"capture_{NAMES[mode]}.png", out)
        print("已存檔")

cap.release()
cv2.destroyAllWindows()
```

這支程式的骨架——讀一影格、依模式處理、顯示、等按鍵——會在第 12 章的視訊處理與第 16 章的 YOLO 即時偵測中原封不動地再出現。建議你每學完一章就回來加一個分支（按 5 顯示直方圖等化、按 6 顯示頻譜、按 7 顯示分割結果），到學期末它就是一台屬於你的即時影像處理實驗機。



圖 2-18　即時處理管線：按鍵切換四種處理模式

2.6.3 案例三：ESP32-CAM 遠端監看

情境：把 ESP32-CAM 放在實驗室或家中，從電腦即時監看，並在偵測到變化時自動存檔。這個案例整合了本章的 2.4.3 節(串流讀取)與後續第 12 章的移動偵測。它的價值在於讓你及早體驗「感測器在遠端、運算在本機」的分散式架構——這正是 2.4.1 節三種架構中「端上取像、近端運算」的實例，也是第 17 章分散式監測網案例的雛形。

2.6.4 案例四：建立你的測試影像庫

情境：承第 1 章 Lab 1-3 的討論，建立一組屬於你自己的標準測試影像，供整學期使用。做法：挑選或拍攝六到十張涵蓋不同特性的影像——平滑漸層、銳利邊緣、細緻紋理、高對比、低光雜訊、含文字、含人臉(需取得同意)、規律圖案。統一尺寸與命名，建立資料夾與說明文件。



圖 2-19　一組為了考驗不同技術而挑選的測試影像

為什麼這值得花時間?因為當你在第 5 章調對比、第 6 章看頻譜、第 7 章去雜訊、第 9 章找邊緣時，都使用同一組影像，你就能跨章比較不同演算法在同一素材上的表現，建立起連貫的直覺。這也呼應第 1 章叫獸開講的反思——選擇測試素材本身，就是一個值得思考的決定。

2.7 本章與全書的觀念連結總表

本章觀念	後續應用的章節	如何延伸
影像即陣列	全書所有章節	所有演算法都是對陣列的運算
uint8 與溢位	第 3、5 章	位元深度、量化;亮度調整需 clip
向量化運算	第 5、6、17 章	濾波效能;邊緣裝置的即時性
BGR 通道順序	第 8 章	色彩空間轉換與 HSV 範圍陷阱
ROI 與遮罩	第 9、10、13 章	車道 ROI、人臉內找眼睛、車牌裁切
合成測試影像	第 4、6、7 章	棋盤格校正、頻譜驗證、雜訊模擬

VideoCapture 迴圈	第 12、16、17 章	視訊處理、即時偵測、邊緣部署的骨架
三種辨識架構	第 10、16、17 章	端上預篩加雲端精算的混合設計
雙平台	第 8、10、12、15-17 章	ESP32-CAM 感測、Jetson 推論與 ROS 2
工程習慣	第 18 章	版本管理、工程筆記、可重現的文件

表 2-2 第 2 章與全書其他章節的觀念連結

程式實作 Lab 2

Lab 2-1 環境建立與驗證

目標：建立可用一整個學期的開發環境。(a)依程式 2-1 建立虛擬環境並安裝套件;(b)執行程式 2-2 驗證安裝，記下三個套件的版本號;(c)產出 requirements.txt 並確認內容;(d)三種開發工具(Jupyter、VS Code、Colab)各試用一次，寫下你覺得各自適合什麼場合;(e)建立本課程的專案資料夾結構(建議：images/ 放素材、labs/ 放各章程式、outputs/ 放結果、utils/ 放共用函式)，並用 git init 開始版本管理。

Lab 2-2 陣列即影像

目標：徹底內化「影像就是陣列」。(a)讀取一張照片，印出 shape、dtype、size，並解釋每個數字的意義;(b)不使用任何影像檔，純以 NumPy 合成：純色塊、水平漸層、垂直漸層、棋盤格、隨機雜訊、以及一個中心亮四周暗的圓形漸層;(c)只用陣列運算(不用 OpenCV 函式)實作：負片、亮度加減、左右翻轉、上下翻轉、裁切中央區域;(d)執行程式 2-5，實測迴圈與向量化的效能差距並記錄倍數;(e)刻意製造一次 uint8 溢位(如對亮部加 100 而不 clip)，觀察結果並解釋為什麼會出現詭異的暗塊。

Lab 2-3 ROI 與遮罩實作

目標：掌握局部操作。(a)用切片取出照片中的一塊區域並存檔;(b)對該區域做模

糊後貼回原圖(製造「馬賽克隱私保護」的效果);(c)驗證「切片是檢視不是複製」：先不加 copy() 修改 ROI，觀察原圖是否被改變，再加上 copy() 重試;(d)建立圓形與多邊形遮罩，只保留遮罩內的影像;(e)進階：用滑鼠事件(cv2.setMouseCallback)讓使用者框選 ROI——這個工具在第 4 章文件掃描與第 12 章物件追蹤都會用到。

Lab 2-4 即時處理管線與 ESP32-CAM

目標：建立整學期最常用的實驗工具。(a)寫一支程式，讀取筆電相機並即時顯示;(b)加入按鍵切換模式(1 原圖、2 灰階、3 邊緣、4 模糊)，並在畫面上顯示目前模式與 FPS;(c)加入按鍵存檔功能(按 s 儲存目前影格);(d)若有 ESP32-CAM，燒錄 CameraWebServer 範例，依程式 2-9 改用串流網址作為輸入，比較兩者的畫質與延遲;(e)把這支程式存成可重複使用的工具(如 live_demo.py)，往後每學一個新演算法就把它加進去——這將是你這門課最有價值的個人資產。

Lab 2-5 效能實測與瓶頸定位

目標：把圖 2-7 的實驗做成你自己的數據，並學會找出瓶頸在哪裡。(a)寫一支程式，對同一張影像分別以 Python 迴圈與 NumPy 向量化做負片，用 time.perf_counter 量測並印出倍數;(b)把影像尺寸由 128×128 逐步放大到 1024×1024，畫出「尺寸對耗時」與「尺寸對倍數」兩條曲線，並解釋為什麼倍數在某個尺寸之後反而下降;(c)量測三種寫法的差異：out = 255 - img（產生新陣列）、np.subtract(255, img, out=out)（就地寫入）、以及 cv2.bitwise_not(img)，說明哪一種最快、為什麼;(d)用 cProfile 或簡單的計時裝飾器，對一條「讀檔→轉灰階→模糊→Canny→存檔」的管線逐步計時，找出最耗時的那一步——結果可能與你的直覺不同;(e)寫下結論：在你的機器上，什麼樣的操作值得優化、什麼不值得?(f)把這支測速程式存進你的工具箱，第 17 章在 Jetson 上部署時會再用到它。

Lab 2-6 打造你的影像處理工具箱

目標：把本章學到的東西整理成一個可重用的模組，供接下來十六章使用。這是

本書最有長期價值的一個 Lab。(a)建立 imtools.py，至少包含四個函式：imread_safe(path)（讀不到就明確拋錯，而不是回傳 None）、debug_info(name, arr)（印出 shape、dtype、值域與平均）、debug_show(name, img)（自動處理灰階 / 彩色與各種值域，見習題 23）、make_test_images()（產生圖 2-6 的六張合成影像）;(b)加上一個 side_by_side(*imgs, titles=None) 函式，把多張影像並排成一張圖，尺寸不同時自動補邊;(c)為每個函式寫一段 docstring 與一個最小測試，確認它在灰階、彩色、float、二值四種輸入下都不會壞;(d)建立 Git 倉庫，寫好 .gitignore 與 README，並以有意義的訊息提交（例如「加入 debug_show，支援 float 輸入」）;(e)寫一支 demo.py，只用 import imtools 就能跑出一條完整的「讀檔→處理→並排顯示」流程;(f)把這個倉庫留著。接下來每一章你都會往裡面加東西——到了第 18 章做專題時，它就是你的個人函式庫。

本章重點整理

- OpenCV 源於 Gary Bradski 1998 年「不要再重造輪子」的構想，1999 年英特爾決定開源(原計畫為閉源的 CVL),2000 年 6 月於 CVPR 首次公開發表;其寬鬆授權讓商業應用得以自由採用，是它普及的關鍵。
- 虛擬環境為每個專案建立獨立套件空間，requirements.txt 讓環境可被他人重現;開發工具 Jupyter(探索)、VS Code(專案)、Colab(免費 GPU)可依任務並用。
- 影像就是 NumPy 陣列：灰階為二維、彩色為三維(高， 寬， 通道);索引順序為 img[y, x](先列後行)，資料型別多為 uint8(0 至 255)，超出範圍會溢位需以 clip 處理。
- 向量化運算比 Python 迴圈快數百倍，因 NumPy 與 OpenCV 底層以最佳化的 C/C++ 實作;這個習慣在第 17 章邊緣部署的即時系統中尤其關鍵。

- OpenCV 使用 BGR 而非 RGB(早期硬體與 Windows 點陣圖格式的歷史包袱)，與 Matplotlib 互通時須轉換;imread 失敗時安靜回傳 None，務必檢查。
- ROI 以切片實作，但切片是「檢視」非複製，修改會影響原圖，需獨立副本時應用 copy();不規則區域則以遮罩配合 bitwise 運算或布林索引處理。



圖 2-12　遮罩的四個步驟：建立、套用、反轉、布林索引

- 三種 AI 辨識架構：端上(ESP32-CAM，低延遲、保護隱私、算力有限)、近端邊緣(Jetson，算力與即時性平衡)、雲端(算力最強但延遲高);三者可分層協作。學習策略為先在電腦驗證邏輯、再移植目標平台。

習題

1. 承叫獸開講：Gary Bradski 為什麼要創立 OpenCV?英特爾在 1999 年做了什麼關鍵決定，這個決定為何重要?

   提示與參考解答：因為他觀察到電腦視覺領域每個實驗室都在重複實作同樣的基礎功能(讀影像、模糊、找邊緣)，整個領域的能量浪費在「重造輪子」上。1999 年的關鍵決定是：原本計畫做閉源商業產品 CVL，經 Bradski 力主改為開源，並更名 OpenCV(靈感來自 OpenGL)。重要性在於只有開放才能真正達成共用基礎、讓領域一起前進;而不強制開源的寬鬆授權，更讓它得以進入無數商業產品。

2. 為什麼要使用虛擬環境?requirements.txt 的用途是什麼?它與第 18 章的哪個要求相關?

提示與參考解答：虛擬環境為每個專案建立獨立的套件空間，避免不同專案需要同一套件的不同版本而衝突。requirements.txt 記錄專案所需的所有套件與版本，讓他人(或日後的自己)能以一行指令完整重現環境。它與第 18 章期末專題技術文件的要求「另一個人照著做能重現你的成果」直接相關。

3. 請說明灰階影像與彩色影像的陣列形狀差異。存取像素時為什麼是 img[y, x] 而非 img[x, y]?

   提示與參考解答：灰階為二維陣列(高度， 寬度);彩色為三維陣列(高度， 寬度， 通道數)。索引為 img[y, x] 是因為 NumPy 陣列的索引順序是「先列(row)後行(column)」，而列對應影像的垂直方向(y)、行對應水平方向(x)，與數學習慣的 (x, y) 相反——這是初學者最常見的錯誤來源之一。

4. 為什麼要避免用 Python 迴圈逐像素處理影像?請估算一張 1920×1080 彩色影像用迴圈與向量化處理的差距量級，並說明原因。

   提示與參考解答：因為向量化比迴圈快數百倍。一張 1920×1080×3 約六百二十萬個元素，Python 迴圈每次疊代都有直譯器的額外開銷;而 NumPy 與 OpenCV 的運算底層以最佳化的 C/C++ 實作、且能利用 SIMD 指令與快取。實測差距常達數百倍。這個習慣在第 17 章邊緣裝置的即時系統上格外關鍵——一個寫成迴圈的前處理就足以讓系統掉影格。

5. 什麼是 uint8 溢位?對一個值為 250 的像素加上 50 會得到什麼?正確做法是什麼?

   提示與參考解答：uint8 範圍是 0 至 255，超出會「繞回」(模 256)。250 + 50 = 300，溢位後得到 44——一個原本很亮的像素反而變暗，產生詭異的暗塊。正確做法：先轉成較大的型別(如 int16 或 float)運算，再用 np.clip(結果， 0, 255) 限制範圍，最後轉回 uint8。第 5 章的亮度調整會反覆用到。

6. OpenCV 為什麼使用 BGR 而非 RGB?這個歷史包袱會在什麼情況下造成問題?這給了我們什麼工程啟示?

   提示與參考解答：因為 OpenCV 早期(2000 年代初)許多相機硬體與 Windows 點陣圖格式採用 BGR 排列，為效能而沿用;待 RGB 成為主流時，已有大量既有程式碼依賴 BGR，改動代價太高。問題常出現在與 Matplotlib、PIL 等使用 RGB 的函式庫互通時，紅藍會顛倒。工程啟示：早期的設計決定會長期影響一個系統，即使當初的

理由早已消失——做架構決定時需考慮長遠影響。

7. imread 讀取失敗時會發生什麼?為什麼這個行為特別容易造成困擾?應如何防範?

   提示與參考解答：它不會拋出例外，而是安靜地回傳 None。困擾在於程式不會在出錯的地方停下，而是在後續操作(如 img.shape)才報出難以理解的錯誤訊息，不易追查根源。防範：讀取後立即檢查 if img is None 並拋出明確的錯誤。另外中文路徑在某些系統會失敗，建議檔名與資料夾都用英文。

8. 什麼是 ROI?為什麼說「切片是檢視不是複製」?這個特性何時是優點、何時是陷阱?

   提示與參考解答：ROI 是感興趣區域，以 NumPy 切片取出影像的局部(如 img[100:300, 200:500])。切片回傳的是原陣列的視圖而非副本，修改它會直接改到原影像。優點：不必複製資料，效能好，且能方便地「處理後貼回」。陷阱：當你只想取出區域來分析、卻不小心改到原圖時，會產生難以察覺的 bug。需要獨立副本時務必呼叫 copy()。

9. 請說明端上、近端邊緣、雲端三種 AI 辨識架構的差異，並舉一個「分層協作」的實例。

   提示與參考解答：端上：在感測裝置本身運算(如 ESP32-CAM)，延遲最低、不需網路、隱私最佳，但算力極有限。近端邊緣：在鄰近的較強裝置運算(如 Jetson)，算力與即時性平衡、資料不出廠區。雲端：遠端資料中心，算力最強但延遲高、需網路、有隱私考量。分層協作實例(第 10 章門禁):ESP32-CAM 端上判斷「有沒有人臉」(高頻低算力)，偵測到才上傳雲端做「是誰」的辨識(低頻高算力)。

10. 本課程為什麼建議「先在電腦上驗證邏輯、再移植到目標平台」?

    提示與參考解答：因為在個人電腦上開發與除錯遠比在嵌入式裝置上容易：有完整的除錯器、可即時顯示中間結果、修改後立即執行不需重新燒錄、且不受裝置算力與記憶體限制。先確認演算法邏輯正確，移植時就只需處理硬體相關的問題(效能、記憶體、介面)，能大幅縮短開發時間。這是嵌入式與機器人開發的標準工作方式。

11. 影像處理的除錯有什麼特殊性?請說明三個實用的除錯技巧。

    提示與參考解答：特殊性：錯誤往往不拋出例外，而是「結果看起來怪怪的」，不

易定位。三個技巧：一、隨時印出陣列的 shape 與 dtype(大多數錯誤源於形狀不符或型別溢位)，可寫成 debug_info 輔助函式;二、視覺化除錯——把中間結果存檔或並排顯示，不要一路算到底才看;三、用自己合成的簡單影像(純色塊、漸層、棋盤格)測試，因為你知道正確答案該長什麼樣。

12. (綜合題)承叫獸開講「不要再重造輪子」：請說明這個原則在你的學習與專題中該如何實踐，以及它有沒有需要注意的界線。

    提示與參考解答：實踐：寫程式前先查是否已有成熟的函式庫或實作(OpenCV 兩千多個演算法、PyPI 生態系)，把時間留給真正需要創造的部分;把自己常用的處理抽成共用模組供各章重複使用;善用開源社群的成果並回饋。界線：一、學習階段有時值得「重造一次輪子」以理解原理(如本書多章要求手動實作演算法再與函式庫對照);二、需確認授權條款是否允許你的用途(對照第 13 章 SIFT 專利的故事);三、直接使用而不理解，遇到問題時無從診斷——理解與效率需要平衡。

13. 計算題：一張 1920×1080 的彩色影像以 uint8 儲存，未壓縮時佔多少記憶體?若轉成 float32 做運算，又會佔多少?若要同時保留原圖、灰階版與三個中間結果（皆為 float32），總共需要多少?

    提示與參考解答：uint8 彩色：1920 × 1080 × 3 × 1 位元組 ≈ 6.22 MB。轉 float32 後每個元素 4 位元組，變成約 24.9 MB，整整四倍。灰階 float32 為 1920 × 1080 × 4 ≈ 8.3 MB;三個彩色 float32 中間結果為 3 × 24.9 ≈ 74.6 MB;加上原圖 6.2 MB，合計約 113 MB。這個數字值得記住：在第 17 章只有 4 GB 記憶體的 Jetson 上同時跑多路相機時，「不必要的中間結果」是最常見的記憶體殺手。務實的做法是盡量就地運算（out 參數）、用完的中間結果及早釋放、以及只在真正需要小數精度的步驟才轉 float。

14. 承圖 2-7：為什麼 NumPy 的向量化會比 Python 迴圈快上千倍?請從「直譯器開銷」與「記憶體存取」兩個角度說明。另外，為什麼在最大的那張影像上，倍數反而比中等尺寸時低?

    提示與參考解答：兩個原因。其一是直譯器開銷：Python 的每次迴圈疊代都要做型別檢查、物件裝箱拆箱與邊界檢查，這些成本遠大於「255 減一個數字」本身;NumPy 把整個迴圈下放到預先編譯好的 C 程式碼，一次付出一份開銷。其二是記憶體存取：NumPy 陣列在記憶體中是連續的，CPU 可以一次載入一整條快取線

> *並以 SIMD 指令同時處理多個元素，而 Python 的迴圈逐一存取、無法向量化。至於大影像倍數下降，是因為向量化版本此時已經快到「瓶頸不在運算而在把資料搬進搬出記憶體」——也就是受限於記憶體頻寬。這個現象在第 17 章的邊緣裝置上更明顯，因為嵌入式平台的記憶體頻寬通常遠低於桌機。*

15. 承圖 2-8：請說明為什麼 img + 50 會讓亮部變黑，而 cv2.add(img, 50) 不會。若要把一張影像的亮度降低 50，有沒有對稱的陷阱?該怎麼寫才安全?

> *提示與參考解答：NumPy 對 uint8 採「繞回」（模 256）運算，250 + 50 = 300 會變成 44，於是最亮的區域變成接近黑色。cv2.add 採「飽和」運算，超過 255 就停在 255，所以不會出現黑點。減法有完全對稱的陷阱：30 − 50 = −20，繞回後變成 236，最暗的區域反而變成接近白色——這在第 7 章做影像相減找差異時特別容易中招。安全的寫法有三種：用 cv2.add / cv2.subtract;或先轉成有號的較寬型別再 np.clip 回來，如 np.clip(img.astype(np.int16) - 50, 0, 255).astype(np.uint8);或整條管線都以 float32 運算，最後一步才轉回 uint8。本書後續採第三種做法的場合最多，因為它同時避免了中間步驟的精度損失。*

16. 承圖 2-11：請解釋「切片是檢視不是複製」。下列三行程式分別會不會動到原圖?(a) roi = img[0:100, 0:100]; roi += 10　(b) roi = img[0:100, 0:100].copy(); roi += 10　(c) roi = img[0:100, 0:100]; roi = roi + 10

> *提示與參考解答：(a) 會。roi 是原陣列的視圖，+= 是就地運算，直接寫回原本的記憶體。(b) 不會。copy() 配置了新的記憶體，兩者從此無關。(c) 不會，但理由與 (b) 不同：roi + 10 產生一個全新的陣列，再把名字 roi 綁到新陣列上，原本的視圖只是被改了名字，原圖沒被碰到。這題的關鍵在於分辨「就地運算」（+=、[:] =、cv2 的 dst 參數）與「產生新物件的運算」（+、np.add 不帶 out）。判準：如果你希望原圖保持不變，就加 .copy();如果你是刻意要改原圖的某一塊（例如打馬賽克），視圖反而讓你省去「貼回去」這一步。*

17. 承 2.4 節：請為下列三個情境各選一種架構（端上 / 近端邊緣 / 雲端），並說明理由：(a) 高鐵月台的跌倒偵測;(b) 病理切片的腫瘤分級;(c) 自家門口的訪客通知。每一題請至少指出一個「選錯架構會出什麼事」。

> *提示與參考解答：本題無唯一答案，評分重點在理由的完整性。一組合理的選擇：(a) 近端邊緣。跌倒必須在數百毫秒內示警，雲端的往返延遲與斷線風險不可接受;且*

月台影像涉及大量乘客隱私，不宜離開站體。選雲端的後果是：網路一塞，警報就慢了，而這正是最需要它的時候。(b) 雲端。切片影像可達數 GB、模型極大，且屬非即時分析，算力才是瓶頸;選端上的後果是根本跑不動，但要注意病歷屬特種個資，上傳前的去識別化與法遵是前提。(c) 端上為主。隱私最敏感（自家影像）、事件稀疏、算力需求低，ESP32-CAM 在端上判斷「有沒有人」即可，只把有人的那幾張送出（圖 2-14 的分層協作）;選雲端的後果是把自家門口的連續影像交給第三方，而且斷網時完全失效。

18. 承 2.5.1 節：以下三個除錯情境，請指出最可能的原因與檢查方法：(a) 用 cv2.imshow 顯示一張剛算好的影像，整個視窗全白;(b) 程式在 img.shape 這一行報 AttributeError;(c) 兩張照片相減後，結果幾乎全黑但偶有極亮的雜點。

    提示與參考解答：(a) 最可能是 dtype 與值域不匹配：若陣列是 float 且值域在 0~1，cv2.imshow 會把 1.0 以上全當成白色;或值域是 0~255 但型別已成 float64。檢查：印出 dtype、min、max，必要時用 cv2.normalize 或 .astype(np.uint8) 修正。(b) img 是 None——imread 找不到檔案時不拋例外而是安靜回傳 None，於是錯誤延後到使用它的地方才爆。檢查：在 imread 後立刻加 if img is None: raise。(c) 幾乎全黑代表兩張確實很像（正常），極亮雜點則是 uint8 減法的負值繞回（第 15 題的對稱陷阱）。檢查：改用 cv2.absdiff，或先轉 int16 再取絕對值。這三題涵蓋了影像處理除錯的三大類：值域、None、型別溢位。

19. 比較 cv2.imshow 與 matplotlib 的 plt.imshow，列出至少三項差異，並說明各自適合在什麼場合使用。

    提示與參考解答：至少三項：其一，通道順序：cv2.imshow 預期 BGR，plt.imshow 預期 RGB，互通時必須 cvtColor（圖 2-9）。其二，灰階影像：plt.imshow 預設套用彩色對應表（viridis），要顯示真正的灰階必須指定 cmap="gray"，而且還要留意 vmin/vmax——未指定時 matplotlib 會自動拉伸對比，讓你誤以為影像比實際更有層次。其三，視窗行為：cv2.imshow 需要搭配 waitKey 才會更新畫面，適合即時影像的連續播放;plt.imshow 產生靜態圖，適合並排比較與輸出到報告。場合：即時攝影機、按鍵互動用 cv2;報告、論文、Jupyter 中的多圖比較用 matplotlib。另外在 Colab 上 cv2.imshow 無法使用，須改用 google.colab.patches 的 cv2_imshow，或直接改用 matplotlib。

20. 什麼是 NumPy 的廣播（Broadcasting）?請說明 img * 1.2、img - img.mean(axis=(0,1))、以及 img * np.array([0.5, 1.0, 1.5]) 這三個運算各自發生了什麼事，並指出第三個運算在影像處理上的實際用途。

    *提示與參考解答：廣播是 NumPy 在形狀不同的陣列間自動對齊維度的規則：從最後一個維度往前比對，兩者相等或其中之一為 1 才能廣播。img * 1.2：純量被廣播到每一個元素，整體變亮兩成。img - img.mean(axis=(0,1))：mean 的結果形狀是 (3,)，對 (H, W, 3) 廣播後等於「每個通道各自減去自己的平均」，這是第 14 至 16 章做資料正規化的標準動作。img * np.array([0.5, 1.0, 1.5])：三個係數分別乘到 B、G、R 三個通道，用途是白平衡與色溫調整（第 8 章）——把藍色壓低、紅色拉高，影像就會偏暖。注意三者都會讓結果變成 float，最後務必 np.clip 並轉回 uint8。*

21. 承 2.3.4 節：什麼是記憶體的連續性（contiguous）?為什麼 img[:, :, ::-1] 做 BGR→RGB 很快，但某些 OpenCV 函式拿到它卻會報錯或變慢?該怎麼處理?

    提示與參考解答：NumPy 以「起點 + 步幅（stride）」描述陣列如何對應到一塊連續的記憶體。img[:, :, ::-1] 只是把最後一維的步幅改成負的，完全沒有搬動任何資料，所以幾乎是零成本——這正是它比 cv2.cvtColor 快的原因。但這樣得到的陣列不是 C 連續的，而 OpenCV 的底層是 C++ 實作、多數函式要求輸入在記憶體中連續排列，因此可能拋出錯誤，或由綁定層偷偷複製一份（於是你省下的時間又還回去了）。處理方式是用 np.ascontiguousarray(img[:, :, ::-1]) 明確複製一份，或乾脆用 cv2.cvtColor。判準是：只在 NumPy 之內運算時用切片反轉最划算;要把結果交給 OpenCV 或寫入檔案時，先確保它是連續的。可用 img.flags["C_CONTIGUOUS"] 檢查。

22. 承 2.5.3 節與圖 2-16：請列出三件「絕對不該提交到 Git」的東西，並說明理由。另外說明 .gitignore 的用途，以及它與 requirements.txt 在「讓別人重現你的專案」這件事上如何分工。

    提示與參考解答：三件不該提交的東西：其一，大型資料集與影像素材——Git 為文字設計，二進位大檔會讓倉庫迅速膨脹且無法有效比對差異，應改用資料集下載腳本或 Git LFS。其二，訓練好的模型權重檔，理由同上（第 15、16 章特別容易犯）。其三，金鑰、密碼與 API token——一旦提交就永遠留在歷史紀錄裡，即使後來刪除也救不回來，應放在環境變數或不被追蹤的設定檔中。此外，虛擬環境資

料夾（cv_env/）、__pycache__、以及程式的輸出結果也都不該提交。.gitignore 用來宣告「這些路徑不要追蹤」，讓 git status 保持乾淨。分工是：.gitignore 負責「不要帶什麼」，requirements.txt 負責「要補什麼」——兩者合起來，別人 clone 之後才能一行指令重建出可執行的環境。

23. 設計題：請設計一個 debug_show(name, img) 函式，要能自動處理下列情況並正確顯示：(a) 灰階與彩色;(b) uint8 與值域 0~1 的 float;(c) 值域超出範圍的中間結果;(d) 二值遮罩。請說明你的判斷邏輯。

    提示與參考解答：核心是「先判斷，再正規化，最後顯示」。一個合理的邏輯：先印出 shape、dtype、min、max（延續程式 2-10）;接著依 dtype 分流——若是浮點數且 max ≤ 1.0，乘以 255 後轉 uint8;若是浮點數但值域超出 0~255，用 cv2.normalize 線性拉伸到 0~255 並在標題註明「已正規化」（這點很重要，否則你會誤以為原始值域本來就正常）;若是 uint8 則直接顯示。維度方面：二維視為灰階（cmap="gray", vmin=0, vmax=255）;三維且第三維為 3 則視為 BGR，顯示前轉 RGB;若只有 0 與 255 兩種值則標註為二值遮罩。這個函式值得你真的寫出來放進工具箱（Lab 2-6），因為接下來十六章的每一個 Lab 都會用到它。

24. 承 2.6.4 節與圖 2-19：為什麼建立一組「自己的標準測試影像」值得花時間?請說明挑選時應涵蓋哪些特性，並舉出兩個「只用網路名圖不做自己的測試影像」會遇到的問題。

    提示與參考解答：價值在於可比較性：整學期用同一組素材，你才能回答「第 5 章的 CLAHE 與第 7 章的去雜訊，在同一張暗部照片上誰比較有效」這類跨章問題;而且每一張都是為了考驗某件事而存在，出問題時容易定位。應涵蓋的特性至少有：平滑漸層（測灰階轉換）、銳利邊緣與細密紋理（測銳化與頻譜）、高雜訊與暗部（測去雜訊）、低對比（測直方圖處理）、規則紋理（測陷波與 AOI）、多個可數的物件（測分割與連通元件）。兩個問題：其一，網路名圖多半已經過壓縮與銳化處理，你量到的是「別人處理後的結果」而非真實感測資料;其二，它們與你未來實際要處理的場景（你的相機、你的打光、你的產品）差距極大，調好的參數移植過去往往完全失效——這正是第 18 章專題最常踩的坑。

25. (綜合設計題)請為本課程設計一份「專案起手式」檢查清單，涵蓋環境、程式組織、資料管理與除錯四個面向，每個面向至少三項。接著說明：如果只能保留其中三項，你會留哪三項?為什麼?

    提示與參考解答：本題無標準答案，評分重點在取捨的理由。一份合理的清單——環境：建立虛擬環境、產生 requirements.txt、記錄 Python 與 OpenCV 版本。程式組織：把常用函式抽成模組（Lab 2-6）、所有路徑寫成常數集中在檔案開頭、主流程寫在 if __name__ == "__main__" 之內。資料管理：原始資料唯讀且永不覆寫、輸出寫到獨立資料夾、資料夾結構固定為 raw / processed / output。除錯：每個步驟後印 shape 與 dtype、把中間結果存檔或並排顯示、隨機過程固定亂數種子以便重現。若只能留三項，一個站得住腳的選擇是：(1) 原始資料唯讀——因為覆寫原始資料是唯一無法復原的錯誤;(2) 虛擬環境加 requirements.txt——因為環境壞掉會讓其他所有努力都無法重現;(3) 每步印 shape 與 dtype——因為它以最低的成本攔截了最大比例的錯誤。其餘項目雖然有用，但都還有補救餘地。

參考資料與延伸閱讀

- OpenCV 官方文件與教學：https://docs.opencv.org(函式參考與 tutorial)
- OpenCV Anniversary — 專案發展歷程：https://opencv.org/anniversary/(本章叫獸開講之史料來源)
- Bradski, G., & Kaehler, A., Learning OpenCV: Computer Vision with the OpenCV Library, O'Reilly, 2008.(創始者親撰的經典入門書)
- NumPy 官方文件：https://numpy.org/doc/(陣列運算與廣播機制)
- Espressif ESP32-CAM 官方範例與文件：https://www.espressif.com
- NVIDIA Jetson 開發者資源：https://developer.nvidia.com/embedded-computing
- 補充教材與課程資訊：@prof(https://page.line.me/prof)